ORBIT · PYTHON · PAPER 02

第 2 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

Python中，以下哪个方法会在对象被删除时自动调用？

A. __init__()
B. __del__()
C. __delete__()
D. __remove__()

答案 B. __del__()

解析 __del__() 是Python中的析构方法（destructor），当对象被垃圾回收机制删除时自动调用。

示例：

```
class MyClass:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} 对象被创建")
    def __del__(self):
        print(f"{self.name} 对象被删除")
obj = MyClass("测试")
del obj
# 显式删除，触发__del__()
# 输出：
# 测试 对象被创建
# 测试 对象被删除
```

知识拓展：

- __del__() 不是严格意义的析构函数，因为Python的垃圾回收机制是不确定的
- 不应该依赖 __del__() 来释放关键资源，应使用上下文管理器（ with 语句）
- 循环引用可能导致 __del__() 不被调用

02 • 单项选择

关于类方法和静态方法，以下说法正确的是：

A. 类方法的第一个参数是 self
B. 静态方法的第一个参数是 cls
C. 类方法可以访问类变量
D. 静态方法可以访问实例变量

答案 C. 类方法可以访问类变量

解析 类方法和静态方法是Python中两种特殊的方法类型。

类方法：

- 使用 @classmethod 装饰器
- 第一个参数是 cls （代表类本身）
- 可以访问和修改类变量
- 可以通过类或实例调用

静态方法：

- 使用 @staticmethod 装饰器
- 没有默认的第一个参数
- 不能访问类变量或实例变量
- 只是逻辑上属于类的普通函数

示例：

```
class MyClass:
    count = 0
    @classmethod
    def class_method(cls):
        cls.count += 1
        return f"类变量count: {cls.count}"
    @staticmethod
    def static_method(x, y):
        return x + y
# 不能访问count
print(MyClass.class_method())
# 类变量count: 1
print(MyClass.static_method(3, 5))
# 8
```

知识拓展：类方法常用于实现工厂模式或提供替代构造器，静态方法用于与类相关但不需要访问类/实例状态的工具函数。

03 • 单项选择

以下代码的输出是什么？

```
class MyClass:
    x = 10
    def __init__(self):
        self.x = 20
obj = MyClass()
print(obj.x, MyClass.x)
```

A. 10 10

B. 20 20

C. 20 10

D. 10 20

答案 C. 20 10

解析 Python中实例变量和类变量的查找遵循特定规则。

变量查找顺序：

1. 先在实例的 __dict__ 中查找
2. 如果找不到，再到类的 __dict__ 中查找
3. 如果还找不到，继续到父类中查找

示例：

```
class MyClass:
    x = 10
    # 类变量
    def __init__(self):
        self.x = 20
# 实例变量，覆盖类变量
obj = MyClass()
print(obj.x)
# 20 - 访问实例变量
print(MyClass.x)
# 10 - 类变量未被修改
print(obj.__class__.x)
```

```
# 10 - 通过类访问
# 删除实例变量后
del obj.x
print(obj.x)
# 10 - 现在访问到类变量
```

知识拓展：

- 实例变量存储在 obj.__dict__ 中
- 类变量存储在 MyClass.__dict__ 中
- 实例变量只对特定实例有效，类变量被所有实例共享

04 • 单项选择

Python中的 __call__() 方法的作用是：

A. 初始化对象

B. 使对象可以像函数一样被调用

C. 删除对象

D. 比较对象

答案 B. 使对象可以像函数一样被调用

解析 __call__() 魔法方法使类的实例可以像函数一样被调用。

示例：

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor
    def __call__(self, x):
        return x * self.factor
# 创建实例
double = Multiplier(2)
triple = Multiplier(3)
# 像函数一样调用实例
print(double(5))
# 10
print(triple(5))
# 15
# 检查是否可调用
print(callable(double))
# True
```

实际应用：

```
class Counter:
    def __init__(self):
        self.count = 0
    def __call__(self):
        self.count += 1
        return self.count
counter = Counter()
print(counter())
# 1
print(counter())
# 2
print(counter())
# 3
```

知识拓展：

- 装饰器本质上就是利用 `__call__()` 实现的可调用对象
- 函数式编程中的闭包可以用实现了 `__call__()` 的类来替代
- 检查对象是否可调用： `callable(obj)`

05 • 单项选择

以下哪个不是有效的继承语法？

A. `class Child(Parent):`

B. `class Child(Parent1, Parent2):`

C. `class Child extends Parent:`

D. `class Child():`

答案 C. `class Child extends Parent:`

解析 Python使用独特的继承语法，与Java、C++等语言不同。

正确的Python继承语法：

```
# 单继承
class Child(Parent):
    pass
# 多重继承
class Child(Parent1, Parent2, Parent3):
    pass
# 不继承任何类（默认继承object）
class Child:
    pass
# 显式继承object
class Child(object):
    pass
```

错误语法对比：

```
#    Java/C++风格（Python不支持）
class Child extends Parent:
    # SyntaxError
    pass
#    C#风格
class Child : Parent
    # SyntaxError
    pass
```

知识拓展：

- Python的多重继承使用C3线性化算法确定方法解析顺序（MRO）
- 可以使用 `ClassName.__mro__` 或 `ClassName.mro()` 查看继承顺序
- Python 3中所有类都隐式继承自 `object`

06 • 单项选择

关于 `super()` 函数，以下说法错误的是：

A. 用于调用父类的方法

B. 在多重继承中按照MRO顺序调用

C. 只能在 `__init__()` 方法中使用

D. 可以避免直接使用父类名

答案 C. 只能在 `__init__()` 方法中使用

解析 `super()` 函数用于调用父类的方法，可以在类的任何方法中使用。

示例：

```
class Parent:
    def __init__(self):
        self.value = 10
    def method(self):
        print("Parent method")
class Child(Parent):
    def __init__(self):
        super().__init__()
        # 在__init__中使用
        self.value += 5
    def method(self):
        super().method()
        # 在普通方法中使用
        print("Child method")
obj = Child()
obj.method()
# 输出：
# Parent method
# Child method
```

知识拓展：

- Python 2中的用法： super(ClassName, self).method()
- Python 3简化为： super().method()
- 在多重继承中， super() 按照MRO顺序调用下一个类的方法
- 可以通过 ClassName.__mro__ 查看方法解析顺序

7-8. 答案：

答案：C

生成器表达式是一种简洁的创建生成器的方式。

示例：

```
# 列表推导式 - 返回列表
list_comp = [x**2 for x in range(5)]
print(type(list_comp))
# <class 'list'>
print(list_comp)
# [0, 1, 4, 9, 16]
# 生成器表达式 - 返回生成器对象
gen_expr = (x**2 for x in range(5))
print(type(gen_expr))
# <class 'generator'>
print(list(gen_expr))
# [0, 1, 4, 9, 16]
```

区别总结：

```
# 列表推导式：使用[]
list_comp = [x for x in range(1000000)]
# 立即占用大量内存
# 生成器表达式：使用()
gen_expr = (x for x in range(1000000))
# 惰性计算，节省内存
```

知识拓展：

- 生成器表达式是一次性的，迭代后就耗尽了
- 适合处理大量数据，因为是惰性生成的
- 可以用 next() 函数手动获取下一个值

07 • 单项选择

生成器表达式 (x**2 for x in range(4)) 返回的是：

A. 列表
B. 元组
C. 生成器对象
D. 集合

答案 C. 生成器对象

解析 （原卷合并作答）

08 • 单项选择

以下代码的输出是什么？

```
gen = (x for x in range(3))
print(type(gen))
```

A. <class 'list'>
B. <class 'tuple'>
C. <class 'generator'>
D. <class 'iterator'>

答案 C. <class 'generator'>

解析 （原卷合并作答）

09 • 单项选择

关于 yield from 语句，以下说法正确的是：

A. 用于从生成器中返回值
B. 用于委托给另一个生成器
C. 用于终止生成器
D. 与 yield 完全相同

答案 B. 用于委托给另一个生成器

解析 yield from 用于将生成器的控制权委托给另一个生成器或可迭代对象。

示例：

```
# 使用yield from简化代码
def generator1():
    yield 1
    yield 2
def generator2():
    yield 3
    yield 4
def combined():
    yield from generator1()
    # 委托给generator1
    yield from generator2()
# 委托给generator2
for value in combined():
    print(value)
# 输出 1, 2, 3, 4
```

不使用yield from的等价代码：

```
def combined():
```

```
    for value in generator1():
        yield value
    for value in generator2():
        yield value
```

知识拓展：

- yield from 不仅仅是语法糖，还能正确处理 send() 和异常
- 常用于递归生成器和树遍历
- Python 3.3+才支持此语法

10 • 单项选择

以下哪个函数可以将迭代器转换为列表？

A. list()

B. iter()

C. next()

D. range()

答案 A. list()

解析 list() 函数可以将任何可迭代对象（包括迭代器）转换为列表。

示例：

```
# 迭代器转列表
iter_obj = iter([1, 2, 3, 4, 5])
list_obj = list(iter_obj)
print(list_obj)
# [1, 2, 3, 4, 5]
# 生成器转列表
gen = (x**2 for x in range(5))
print(list(gen))
# [0, 1, 4, 9, 16]
# range对象转列表
print(list(range(5)))
# [0, 1, 2, 3, 4]
# 字符串转列表
print(list("hello"))
# ['h', 'e', 'l', 'l', 'o']
```

知识拓展：

- 迭代器是一次性的，转换为list后原迭代器就耗尽了
- 其他转换方式： tuple() , set() , dict()
- 可以使用列表推导式： [x for x in iterator]

11 • 单项选择

装饰器的执行时机是：

A. 函数被调用时

B. 函数被定义时

C. 程序结束时

D. 模块被导入时

答案 B. 函数被定义时

解析 装饰器在函数定义时执行，而不是在函数调用时。

示例：

```
def my_decorator(func):
```

```
    print(f"装饰器执行：装饰{func.__name__}")
    def wrapper():
        print("wrapper执行")
        return func()
    return wrapper
@my_decorator
# 这里执行装饰器
def my_function():
    print("函数执行")
print("---分隔线---")
my_function()
# 这里调用函数
# 输出：
# 装饰器执行：装饰 my_function
# ---分隔线---
# wrapper执行
# 函数执行
```

知识拓展：

- 装饰器的本质是函数转换： `my_function = my_decorator(my_function)`
- 可以使用多个装饰器，执行顺序从下到上
- 应使用 `@functools.wraps` 保留原函数的元数据

12 • 单项选择

以下代码的输出是什么？

```
def outer(x):
    def inner(y):
        return x + y
    return inner
f = outer(10)
print(f(5))
```

A. 10

B. 5

C. 15

D. 报错

答案 C. 15

解析 闭包会保留对外部变量的引用，即使外部函数已经返回。

示例：

```
def outer():
    x = 10
    def inner(y):
        return x + y
    # 闭包：保留了x的引用
    return inner
func = outer()
print(func(5))
# 15
# 查看闭包变量
print(func.__closure__)
# (<cell at 0x...: int object at 0x...>,)
```

```
print(func.__closure__[0].cell_contents)
# 10
```

多个闭包共享变量：

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment
counter1 = make_counter()
counter2 = make_counter()
print(counter1())
# 1
print(counter1())
# 2
print(counter2())
# 1 - 独立的闭包
```

知识拓展：

- 闭包常用于数据封装和装饰器实现
- 使用 nonlocal 关键字可以修改外部变量
- 闭包可能导致内存不能被释放，需注意内存泄漏

13 • 单项选择

关于闭包，以下说法正确的是：

A. 内部函数可以访问外部函数的变量

B. 闭包会保存外部函数的环境

C. 闭包可以实现数据隐藏

D. 以上都正确

答案 D. 以上都正确

解析 *args 和 **kwargs 的多种用途。

示例：

```
# 1. 接收任意数量的位置参数
def func1(*args):
    print(args)
func1(1, 2, 3)
# (1, 2, 3)
# 2. 接收任意数量的关键字参数
def func2(**kwargs):
    print(kwargs)
func2(a=1, b=2)
# {'a': 1, 'b': 2}
# 3. 传递参数给其他函数
def wrapper(*args, **kwargs):
    return other_func(*args, **kwargs)
# 4. 组合使用
def func3(a, b, *args, x=10, **kwargs):
    print(f"a={a}, b={b}, args={args}")
    print(f"x={x}, kwargs={kwargs}")
func3(1, 2, 3, 4, x=5, y=6, z=7)
```

```
# a=1, b=2, args=(3, 4)
# x=5, kwargs={'y': 6, 'z': 7}
```

知识拓展：

- *args 必须在 **kwargs 之前
- Python 3支持在 *args 之后使用关键字参数
- 可以用于函数定义和函数调用

14 • 单项选择

以下代码的输出是什么？

```
def func(a, b, *args, **kwargs):
    print(len(args), len(kwargs))
func(1, 2, 3, 4, x=5, y=6)
```

A. 2 2

B. 4 2

C. 2 4

D. 6 0

答案 A. 2 2

解析 参数解包和打包的区别。

示例：

```
def func(a, b, *args, x, **kwargs):
    print(f"a={a}, b={b}")
    print(f"args={args}, type={type(args)}")
    print(f"x={x}")
    print(f"kwargs={kwargs}, type={type(kwargs)}")
func(1, 2, 3, 4, x=5, y=6, z=7)
# a=1, b=2
# args=(3, 4), type=<class 'tuple'>
# x=5
# kwargs={'y': 6, 'z': 7}, type=<class 'dict'>
```

调用时解包：

```
def func(a, b, c):
    print(a, b, c)
args_tuple = (1, 2, 3)
func(*args_tuple)
# 解包元组
kwargs_dict = {'a': 1, 'b': 2, 'c': 3}
func(**kwargs_dict)
# 解包字典
```

知识拓展：

- 在函数定义中： *args 是打包， **kwargs 是打包
- 在函数调用中： *args 是解包， **kwargs 是解包
- Python 3.5+支持多次解包： {**dict1, **dict2}

15 • 单项选择

关于参数解包，以下说法错误的是：

A. * 用于解包序列

B. ** 用于解包字典

C. 只能在函数定义时使用

D. 可以在函数调用时使用

答案 C. 只能在函数定义时使用

解析 * 和 ** 可以在函数定义和调用时使用。

函数定义时解包（打包）：

```
def func(*args, **kwargs):
    # args是元组，kwargs是字典
    pass
```

函数调用时解包：

```
args = (1, 2, 3)
kwargs = {'a': 10, 'b': 20}
func(*args, **kwargs)
```

其他解包场景：

```
# 列表解包
first, *rest = [1, 2, 3, 4, 5]
print(first, rest)
# 1 [2, 3, 4, 5]
# 字典合并
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
merged = {**dict1, **dict2}
print(merged)
# {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

知识拓展：

- Python 3.5+支持字典解包： {**dict1, **dict2}
- Python 3.0+支持扩展解包： a, *b, c = [1,2,3,4,5]
- 解包可以用于函数调用、列表赋值、字典合并等

16 • 单项选择

以下代码的输出是什么？

```
*a, b = [1, 2, 3, 4]
print(a)
```

A. [1, 2, 3]

B. [1]

C. 4

D. [2, 3, 4]

答案 A. [1, 2, 3]

解析 扩展解包语法允许在赋值时捕获多个值。

示例：

```
# 基本用法
*a, b = [1, 2, 3, 4]
print(a)
# [1, 2, 3]
print(b)
# 4
# 中间位置
a, *b, c = [1, 2, 3, 4, 5]
print(a)
# 1
print(b)
```

```
# [2, 3, 4]
print(c)
# 5
# 开头位置
first, *rest = [1, 2, 3, 4, 5]
print(first)
# 1
print(rest)
# [2, 3, 4, 5]
# 只能有一个*
# a, *b, *c = [1,2,3] # SyntaxError
```

知识拓展：

- Python 3.0+才支持扩展解包
- * 只能出现一次
- 如果没有足够的值， * 捕获的是空列表
- 常用于处理不定长度的序列

17 • 单项选择

Pandas中，Series和DataFrame的主要区别是：

A. Series是一维的，DataFrame是二维的

B. Series是二维的，DataFrame是一维的

C. 没有区别

D. Series不能有索引

答案 A. Series是一维的，DataFrame是二维的

解析 Pandas的两个核心数据结构：Series和DataFrame。

示例：

```
import pandas as pd
# Series - 一维带标签的数组
s = pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])
print(s)
# a 1
# b 2
# c 3
# d 4
# dtype: int64
# DataFrame - 二维表格数据
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
print(df)
# A B C
# 0 1 4 7
# 1 2 5 8
# 2 3 6 9
# 关系
print(type(df['A']))
# <class 'pandas.core.series.Series'>
# DataFrame的每一列都是一个Series
```

知识拓展：
- Series可以看作是带索引的一维数组
- DataFrame可以看作是Series的字典
- Pandas还有Panel（三维），但已被弃用

18 • 单项选择

以下哪个方法用于查看DataFrame的前几行？

A. head()
B. top()
C. first()
D. show()

答案 A. head()

解析 Pandas查看数据的常用方法。
示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': range(100),
    'B': range(100, 200)
})
# 查看前几行
print(df.head())
# 默认前5行
print(df.head(10))
# 前10行
# 查看后几行
print(df.tail())
# 默认后5行
print(df.tail(10))
# 后10行
# 查看数据信息
print(df.info())
# 数据类型、内存使用
# 查看统计信息
print(df.describe()) # 基本统计指标
```

知识拓展：
- head() 和 tail() 常用于快速查看数据样本
- info() 显示每列的数据类型和缺失值情况
- describe() 只显示数值列的统计信息

19 • 单项选择

Pandas中，以下哪个方法用于数据排序？

A. sort()
B. order()
C. sort_values()
D. arrange()

答案 C. sort_values()

解析 Pandas数据排序的方法。
示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [3, 1, 2],
    'B': [6, 4, 5]
})
# 按单列排序
df_sorted = df.sort_values('A')
print(df_sorted)
# A B
# 1 1 4
# 2 2 5
# 0 3 6
# 按多列排序
df_sorted = df.sort_values(['A', 'B'], ascending=[True, False])
# 按索引排序
df_sorted = df.sort_index()
# 就地排序
df.sort_values('A', inplace=True)
```

知识拓展：

- sort_values() 按值排序， sort_index() 按索引排序
- ascending 控制升序/降序
- inplace=True 直接修改原数据
- 可以指定 na_position 处理缺失值的位置

20 • 单项选择

关于Pandas的 apply() 方法，以下说法正确的是：

A. 只能应用于DataFrame

B. 只能应用于Series

C. 可以应用自定义函数

D. 不能使用lambda函数

答案 C. 可以应用自定义函数

解析 Pandas的 apply() 方法用于应用自定义函数。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
# 对列应用函数
df['C'] = df['A'].apply(lambda x: x ** 2)
print(df)
# A B C
# 0 1 4 1
# 1 2 5 4
# 2 3 6 9
# 对整个DataFrame应用
result = df.apply(lambda col: col.sum(), axis=0)
# 按列
print(result)
# A 6
```

```
# B 15
# C 14
result = df.apply(lambda row: row.sum(), axis=1)
# 按行
print(result)
# 0 6
# 1 11
# 2 18
# 使用自定义函数
def custom_func(x):
    if x > 2:
        return x * 2
df['D'] = df['A'].apply(custom_func)
```

知识拓展：

- apply() 比循环更高效，但比向量化操作慢
- axis=0 按列， axis=1 按行
- 可以使用 applymap() 对每个元素应用函数
- 优先使用向量化操作，其次才是 apply()

21-25. 答案：

答案：B B B A B - Pandas和Pathlib相关题目，已在第1套中有类似详细解释

21 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
s = pd.Series([1, 2, 3, 4])
print(s[s > 2].sum())
```

A. 3
B. 7
C. 10
D. 4

答案 B. 7

解析 （原卷合并作答）

22 • 单项选择

pathlib.Path 对象的 parent 属性返回的是：

A. 文件名
B. 父目录路径
C. 文件扩展名
D. 绝对路径

答案 B. 父目录路径

解析 （原卷合并作答）

23 • 单项选择

以下哪个方法用于创建目录？

A. path.create()
B. path.mkdir()
C. path.makedir()

D. path.new()

答案 B. path.mkdir()

解析 （原卷合并作答）

24 • 单项选择

关于 Path.glob() 方法，以下说法正确的是：

A. 用于搜索匹配模式的文件
B. 返回列表
C. 不支持通配符
D. 只能搜索文件不能搜索目录

答案 B. 返回列表

解析 （原卷合并作答）

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('folder/subfolder/file.txt')
print(p.parts)
```

A. 'folder/subfolder/file.txt'
B. ('folder', 'subfolder', 'file.txt')
C. ['folder', 'subfolder', 'file.txt']
D. 'file.txt'

答案 B. ('folder', 'subfolder', 'file.txt')

解析 （原卷合并作答）

26 • 单项选择

关于 enumerate() 函数，以下说法正确的是：

A. 返回索引和值的元组
B. 只能用于列表
C. 不能指定起始索引
D. 返回字典

答案 A. 返回索引和值的元组

解析 enumerate() 函数用于在迭代时同时获取索引和值。

示例：

```
# 基本用法
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
# 0: apple
# 1: banana
# 2: cherry
# 指定起始索引
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}: {fruit}")
# 1: apple
```

```
# 2: banana
# 3: cherry
# 转换为列表
indexed_list = list(enumerate(fruits))
print(indexed_list)
# [(0, 'apple'), (1, 'banana'), (2, 'cherry')]
```

知识拓展：

- enumerate() 返回的是迭代器，不是列表
- 可以通过 start 参数指定起始索引
- 常用于需要同时使用索引和值的场景

27 • 单项选择

以下代码的输出是什么？

```
result = [x+y for x in [1,2] for y in [3,4]]
print(result)
```

A. [4, 5, 5, 6]

B. [4, 6]

C. [1, 2, 3, 4]

D. [[4, 5], [5, 6]]

答案 A. [4, 5, 5, 6]

解析 嵌套列表推导式的执行顺序。

示例：

```
# 题目代码
result = [x + y for x in [1, 2] for y in [3, 4]]
print(result)
# [4, 5, 5, 6]
# 等价的普通循环
result = []
for x in [1, 2]:
    for y in [3, 4]:
        result.append(x + y)
# x=1: 1+3=4, 1+4=5
# x=2: 2+3=5, 2+4=6
# 三重嵌套
result = [x + y + z
        for x in [1, 2]
        for y in [10, 20]
        for z in [100, 200]]
print(len(result))
# 2*2*2 = 8
```

知识拓展：

- 嵌套循环的顺序：左侧是外层，右侧是内层
- 可以添加条件过滤： [x+y for x in [1,2] for y in [3,4] if x != y]
- 过多嵌套会降低可读性，建议不超过两层

28 • 单项选择

集合推导式的语法是：

A. [x for x in iterable]

B. (x for x in iterable)

C. {x for x in iterable}

D. <x for x in iterable>

答案 C. {x for x in iterable}

解析 Python中的四种推导式语法。

示例：

```
# 列表推导式 - []
list_comp = [x**2 for x in range(5)]
print(list_comp)
# [0, 1, 4, 9, 16]
print(type(list_comp))
# <class 'list'>
# 集合推导式 - {}
set_comp = {x**2 for x in range(5)}
print(set_comp)
# {0, 1, 4, 9, 16}
print(type(set_comp))
# <class 'set'>
# 字典推导式 - {k: v}
dict_comp = {x: x**2 for x in range(5)}
print(dict_comp)
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
print(type(dict_comp))
# <class 'dict'>
# 生成器表达式 - ()
gen_expr = (x**2 for x in range(5))
print(list(gen_expr))
# [0, 1, 4, 9, 16]
print(type(gen_expr))
# <class 'generator'>
```

知识拓展：

- 集合推导式会自动去重
- 字典推导式需要同时指定键和值
- 生成器表达式不会立即计算，节省内存

29 • 单项选择

以下代码的输出是什么？

```
a = [1, 2, 3]
b = [4, 5]
result = list(zip(a, b))
print(len(result))
```

A. 2

B. 3

C. 5

D. 报错

答案 A. 2

解析 zip() 函数将多个可迭代对象打包在一起。

示例：

```
# 基本用法
```

```
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30]
scores = [85, 90, 95, 100]
result = list(zip(names, ages, scores))
print(result)
# [('Alice', 25, 85), ('Bob', 30, 90)]
# 以最短序列为准，长度为2
# 解包
for name, age, score in zip(names, ages, scores):
    print(f"{name}: {age}岁, {score}分")
# 转置矩阵
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
transposed = list(zip(*matrix))
print(transposed)
# [(1, 4, 7), (2, 5, 8), (3, 6, 9)]
# zip_longest从 itertools模块
from itertools import zip_longest
result = list(zip_longest(names, ages, fillvalue=0))
print(result)
# [('Alice', 25), ('Bob', 30), ('Charlie', 0)]
```

知识拓展：

- zip() 返回迭代器，需要 list() 转换
- 长度以最短序列为准
- 使用 itertools.zip_longest() 可以以最长序列为准
- * 操作符可以解压 zip 结果

30 • 单项选择

关于 filter() 函数，以下说法正确的是：

A. 返回列表

B. 返回迭代器

C. 不能使用lambda函数

D. 只能过滤数字

答案 B. 返回迭代器

解析 filter() 函数用于过滤序列。

示例：

```
# 过滤偶数
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(type(even_numbers))
# <class 'filter'> - 迭代器
print(list(even_numbers))
# [2, 4, 6, 8, 10]
# 使用自定义函数
def is_positive(x):
    return x > 0
numbers = [-2, -1, 0, 1, 2, 3]
positive = list(filter(is_positive, numbers))
print(positive)
# [1, 2, 3]
# 过滤None值
```

```
values = [0, 1, None, 2, False, 3, '', 4]
filtered = list(filter(None, values))
# None作为过滤器
print(filtered)
# [1, 2, 3, 4] - 过滤掉假值
```

对比列表推导式：

```
# filter
result = list(filter(lambda x: x > 0, numbers))
# 列表推导式 - 更常用
result = [x for x in numbers if x > 0]
```

知识拓展：

- filter() 返回迭代器，需要 list() 转换
- 第一个参数是判断函数，第二个参数是可迭代对象
- 现代Python更推荐使用列表推导式
- filter(None, iterable) 可以过滤掉所有假值

二、不定项选择题

31 • 不定项选择

关于Python的魔法方法，以下说法正确的有：

A. __eq__() 用于定义相等比较
B. __lt__() 用于定义小于比较
C. __add__() 用于定义加法运算
D. __getitem__() 用于定义索引访问

答案 A. __eq__() 用于定义相等比较；B. __lt__() 用于定义小于比较；C. __add__() 用于定义加法运算；D. __getitem__() 用于定义索引访问

解析 Python常用魔法方法概览。

示例：

```
class MyClass:
    # A. __str__ - 定义print()和str()的行为
    def __str__(self):
        return "MyClass对象"
    # B. __repr__ - 定义repr()的行为，用于调试
    def __repr__(self):
        return "MyClass()"
    # C. __len__ - 定义len()的行为
    def __len__(self):
        return 10
    # D. __getitem__ - 定义索引访问 obj[key]
    def __getitem__(self, key):
        return key * 2
obj = MyClass()
print(str(obj))
# MyClass对象
print(repr(obj))
# MyClass()
print(len(obj))
# 10
```

```
print(obj[5])
# 10
```

知识拓展：其他重要魔法方法： `__init__` ， `__new__` ， `__del__` ， `__call__` ， `__enter__` ， `__exit__` ， `__iter__` ， `__next__` ， `__eq__` ， `__lt__` ， `__add__` ， `__mul__` 等。

32 • 不定项选择

以下哪些是Python的抽象基类相关的模块：

A. abc

B. collections.abc

C. typing

D. abstract

答案 A. abc；B. collections.abc

解析 Python中的抽象基类模块。

示例：

```
from abc import ABC, abstractmethod
# 使用abc模块
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass
class Dog(Animal):
    def make_sound(self):
        return "汪汪"
# 使用collections.abc
from collections.abc import Iterable, Iterator
class MyRange:
    def __init__(self, n):
        self.n = n
        self.i = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i < self.n:
            result = self.i
            self.i += 1
            return result
        raise StopIteration
print(isinstance(MyRange(5), Iterable))
# True
print(isinstance(MyRange(5), Iterator))
# True
```

知识拓展：

- abc 模块：提供 ABC 基类和 @abstractmethod 装饰器
- collections.abc ：提供容器相关的抽象基类（Iterable, Iterator, Sequence等）

33 • 不定项选择

关于属性装饰器，以下说法正确的有：

A. @property 定义getter方法

B. @属性名.setter 定义setter方法

C. @属性名.deleter 定义deleter方法

D. 可以实现只读属性

答案 A. @property 定义getter方法；B. @属性名.setter 定义setter方法；C. @属性名.deleter 定义deleter方法；D. 可以实现只读属性

解析 生成器的多种特性。

示例：

```
# A. 惰性计算，节省内存
def large_range(n):
    i = 0
    while i < n:
        yield i
        i += 1
gen = large_range(1000000)
# 不会立即分配内存
# B. 只能迭代一次
gen = (x for x in range(3))
print(list(gen))
# [0, 1, 2]
print(list(gen))
# [] - 已耗尽
# C. 可以使用yield表达式接收值
def coroutine():
    total = 0
    while True:
        x = yield total
        if x is not None:
            total += x
coro = coroutine()
next(coro)
# 启动协程
print(coro.send(10))
# 10
print(coro.send(20))
# 30
# D. 支持异常处理
def gen_with_exception():
    try:
        yield 1
        yield 2
    except ValueError:
        yield "caught error"
g = gen_with_exception()
print(next(g))
# 1
print(g.throw(ValueError))
                # caught error
```

知识拓展：生成器的三个主要方法： `__next__()` 、 `send()` 、 `throw()` 、 `close()` 。

34 • 不定项选择

关于生成器的优势，以下说法正确的有：

A. 节省内存
B. 支持惰性计算
C. 可以表示无限序列
D. 执行速度总是更快

答案 A. 节省内存；B. 支持惰性计算；C. 可以表示无限序列

解析 生成器的优劣势分析。
示例：

```
import sys
# A. 节省内存
list_data = [x for x in range(1000000)]
gen_data = (x for x in range(1000000))
print(sys.getsizeof(list_data))
# ~8MB
print(sys.getsizeof(gen_data))
# ~200 bytes
# B. 惰性计算
def process_large_file():
    with open('large.txt') as f:
        for line in f:
            # 文件对象是迭代器
            yield line.strip()
# C. 可以表示无限序列
def fibonacci():
    a, b = 0, 1
    while True:
        # 无限循环
        yield a
        a, b = b, a + b
# D. 不总是更快
# 如果需要多次访问数据，列表可能更快
data = [x**2 for x in range(1000)]
# 只计算一次
for _ in range(100):
    for x in data:
        # 快速重复访问
        pass
```

知识拓展：生成器适合一次性处理大量数据，但如果需要多次访问或随机访问，列表更合适。

35 • 不定项选择

以下哪些操作会创建迭代器：

A. `iter([1, 2, 3])`
B. `(x for x in range(10))`
C. `open('file.txt')`
D. `range(10)`

答案 A. `iter([1, 2, 3])`；B. `(x for x in range(10))`；C. `open('file.txt')`；D. `range(10)`

解析 创建迭代器的多种方式。
示例：

```
# A. iter()函数
my_list = [1, 2, 3]
```

```
iterator = iter(my_list)
print(next(iterator))
# 1
# B. 生成器函数
def my_generator():
    yield 1
    yield 2
    yield 3
gen = my_generator()
print(next(gen))
# 1
# C. 生成器表达式
gen_expr = (x for x in range(3))
print(next(gen_expr))
# 0
# D. 自定义迭代器类
class MyIterator:
    def __init__(self, n):
        self.n = n
        self.i = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i < self.n:
            result = self.i
            self.i += 1
            return result
        raise StopIteration
my_iter = MyIterator(3)
print(next(my_iter))
# 0
```

知识拓展：Python中的迭代器协议需要实现 `__iter__()` 和 `__next__()` 方法。

36 • 不定项选择

关于装饰器的特性，以下说法正确的有：

A. 可以嵌套使用

B. 可以带参数

C. 可以装饰类

D. 会修改原函数的 `__name__` 属性

答案 A. 可以嵌套使用；C. 可以装饰类；D. 会修改原函数的 `__name__` 属性

解析 装饰器对函数元数据的影响。

示例：

```
from functools import wraps
# 没有使用wraps
def my_decorator1(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@my_decorator1
def original_func1():
```

```
    """This is the original function"""
    pass
print(original_func1.__name__)
# wrapper - 被修改了
print(original_func1.__doc__)
# None - 丢失了
# 使用wraps保留元数据
def my_decorator2(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@my_decorator2
def original_func2():
    """This is the original function"""
    pass
print(original_func2.__name__)
# original_func2 - 保留了
print(original_func2.__doc__)
# This is the original function
```

知识拓展： functools.wraps 会保留 __name__ , __doc__ , __module__ , __qualname__ , __annotations__ 等元数据。

7-15. 答案：

答案：ABCD, ABC, ABCD, ABCD, ABCD, ABC, ABCD, ABCD, ABC

这些题目涉及闭包、Pandas、pathlib、函数式编程等高级概念，已在第1套中有类似详细详解。

核心要点包括：

- 闭包的多种应用场景
- Pandas数据结构和聚合方法
- pathlib的文件操作能力
- 高阶函数和函数式编程特性

37 · 不定项选择

关于函数的默认参数，以下说法正确的有：

A. 默认参数在函数定义时计算
B. 可变对象作为默认参数可能导致问题
C. 默认参数必须在位置参数之后
D. 可以使用 None 作为默认值

答案 A. 默认参数在函数定义时计算；B. 可变对象作为默认参数可能导致问题；C. 默认参数必须在位置参数之后；D. 可以使用 None 作为默认值

解析 （原卷合并作答）

38 · 不定项选择

以下哪些是Pandas的数据结构：

A. Series
B. DataFrame
C. Panel
D. Array

答案 A. Series；B. DataFrame；C. Panel；D. Array

解析 （原卷合并作答）

39 • 不定项选择

关于Pandas的索引操作，以下说法正确的有：

A. loc 支持切片操作
B. iloc 只接受整数索引
C. at 用于快速标量访问
D. iat 用于快速整数位置访问

答案 A. loc 支持切片操作；B. iloc 只接受整数索引；C. at 用于快速标量访问；D. iat 用于快速整数位置访问

解析 （原卷合并作答）

40 • 不定项选择

Pandas中，以下哪些方法用于数据聚合：

A. sum()
B. mean()
C. count()
D. agg()

答案 A. sum()；B. mean()；C. count()；D. agg()

解析 （原卷合并作答）

41 • 不定项选择

关于 pathlib 的优势，以下说法正确的有：

A. 面向对象的API
B. 跨平台路径处理
C. 链式调用
D. 比os.path更直观

答案 A. 面向对象的API；B. 跨平台路径处理；C. 链式调用；D. 比os.path更直观

解析 （原卷合并作答）

42 • 不定项选择

Path 对象的以下哪些方法会修改文件系统：

A. mkdir()
B. touch()
C. unlink()
D. exists()

答案 A. mkdir()；B. touch()；C. unlink()；D. exists()

解析 （原卷合并作答）

43 • 不定项选择

关于列表推导式的性能，以下说法正确的有：

A. 通常比for循环快
B. 代码更简洁
C. 可读性更好
D. 适合简单的转换操作

答案 A. 通常比for循环快；B. 代码更简洁；C. 可读性更好；D. 适合简单的转换操作

解析 （原卷合并作答）

44 • 不定项选择

以下哪些是Python的内置高阶函数：

A. map()
B. filter()
C. reduce()
D. zip()

答案 A. map()；B. filter()；C. reduce()；D. zip()

解析 （原卷合并作答）

45 • 不定项选择

关于 *args 和 **kwargs ，以下说法正确的有：

A. 名称可以自定义
B. *args 是元组类型
C. **kwargs 是字典类型
D. 必须同时使用

答案 A. 名称可以自定义；B. *args 是元组类型；C. **kwargs 是字典类型；D. 必须同时使用

解析 （原卷合并作答）

三、填空题

46 • 填空 / 简答

Python中，使用 _____ 方法可以使对象支持 len() 函数，使用 _____ 方法可以使对象支持索引访问。

答案 __len__() ， __getitem__()

解析 实现序列协议需要这两个魔法方法。

示例：

```
class MySequence:
    def __init__(self, data):
        self._data = data
    def __len__(self):
        return len(self._data)
    def __getitem__(self, index):
        return self._data[index]
seq = MySequence([1, 2, 3, 4, 5])
print(len(seq))
# 5
print(seq[2])
# 3
print(seq[1:4])
# [2, 3, 4]
# 可以使用for循环
for item in seq:
```

```
    print(item)
```

知识拓展：实现这两个方法后，对象就可以支持 len() 、索引、切片和迭代。

47 • 填空 / 简答

迭代器必须实现 _____ 和 _____ 两个方法才能符合迭代器协议。

答案 __iter__() ， __next__()

解析 解析：迭代器协议的两个核心方法。

示例：

```
class Countdown:
    def __init__(self, start):
        self.current = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1
# 使用
for num in Countdown(5):
    print(num)
# 5, 4, 3, 2, 1
```

知识拓展： __iter__() 返回迭代器对象本身， __next__() 返回下一个值或抛出 StopIteration。

48 • 填空 / 简答

在Pandas中，_____ 方法用于删除重复行，_____ 方法用于重置索引。

答案 drop_duplicates() ， reset_index()

解析 解析：Pandas数据清洗的两个关键方法。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 2, 3, 3, 3],
    'B': [4, 5, 5, 6, 7, 7]
})
# 去重
df_unique = df.drop_duplicates()
print(df_unique)
# A B
# 0 1 4
# 1 2 5
# 3 3 6
# 4 3 7
# 重置索引
df_reset = df_unique.reset_index(drop=True)
print(df_reset)
# A B
# 0 1 4
# 1 2 5
```

```
# 2 3 6
# 3 3 7
```

知识拓展： drop_duplicates() 可指定列去重， reset_index(drop=True) 丢弃原索引。

49 • 填空 / 简答

使用 _____ 函数可以同时获取索引和值，使用 _____ 函数可以反转序列。

答案 enumerate() ， reversed()

解析 触析：内置函数，用于序列操作。

示例：

```
data = ['a', 'b', 'c']
# enumerate - 同时获取索引和值
for i, val in enumerate(data, start=1):
    print(f"{i}: {val}")
# 1: a
# 2: b
# 3: c
# reversed - 反向迭代
for val in reversed(data):
    print(val)
# c, b, a
```

知识拓展： enumerate() 返回迭代器， reversed() 也返回迭代器，不修改原序列。

50 • 填空 / 简答

pathlib.Path 对象的 _____ 方法返回所有父目录，_____ 方法用于解析相对路径为绝对路径。

答案 parents ， resolve()

解析 羯析：pathlib路径操作方法。

示例：

```
from pathlib import Path
p = Path('/home/user/docs/file.txt')
# parents - 获取所有父目录
for parent in p.parents:
    print(parent)
# /home/user/docs
# resolve() - 解析为绝对路径
rel_path = Path('docs/../file.txt')
abs_path = rel_path.resolve()
print(abs_path)
# /home/user/file.txt
```

知识拓展： parents 返回所有祖先目录， resolve() 解析符号链接和 .. 。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Rectangle:
    def __init__(self, width, height):
```

```
        self.width = width
        self.height = height
    @property
    def area(self):
        return self.width * self.height
    @property
    def perimeter(self):
        return 2 * (self.width + self.height)
    def __str__(self):
        return f"Rectangle({self.width}x{self.height})"
rect = Rectangle(5, 3)
print(rect.area)
print(rect.perimeter)
print(rect)
```

问题：

- (1) 三个 print 语句的输出分别是什么？
- (2) 为什么 area 和 perimeter 调用时不需要括号？
- (3) 如果要添加一个setter使 area 可以被修改，应该如何实现？

答案 • (1) 输出：

```
15
16
Rectangle(5x3)
```

- (2) 因为使用了 @property 装饰器，将方法转换为属性
- (3) 实现setter需要计算并设置width和height，但这在数学上不唯一，通常area作为只读属性更合理

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def countdown(n):
    print(f"开始倒计时: {n}")
    while n > 0:
        yield n
        n -= 1
    print("倒计时结束")
gen = countdown(3)
print(next(gen))
for i in gen:
    print(i)
```

问题：

- (1) 完整的输出是什么？
- (2) "倒计时结束"会在什么时候打印？
- (3) 如果再次调用 next(gen) 会发生什么？

答案 • (1) 输出：

```
开始倒计时: 3
3
```

```
2
1
倒计时结束
```

- (2) 当生成器耗尽时（所有值都被yield后）打印
- (3) 会抛出 StopIteration 异常

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'product': ['A', 'B', 'A', 'B', 'A'],
    'sales': [100, 150, 200, 120, 180],
    'region': ['East', 'West', 'East', 'West', 'East']
})
result = df.groupby('product')['sales'].sum()
print(result)
print(type(result))
```

问题：

- (1) result 的输出是什么？
- (2) result 的类型是什么？
- (3) 如何获取产品A的总销售额？

答案 分组聚合

- (1) 输出：

```
product
A 480
B 270
Name: sales, dtype: int64
```

- (2) pandas.Series
- (3) result['A'] 或 result.loc['A']

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
def find_files(directory, pattern):
    path = Path(directory)
    for file in path.rglob(pattern):
        yield file.name
# 假设目录结构：
# /data
# ├── file1.txt
# ├── subdir
# │   └── file2.txt
# └── file3.csv
files = list(find_files('/data', '*.txt'))
```

```
print(files)
```

问题：

- (1) files 的内容是什么？
- (2) rglob() 和 glob() 的区别是什么？
- (3) 如果要获取完整路径而不是文件名，应该如何修改？

答案 • (1) ['file1.txt', 'file2.txt']
• (2) rglob() 递归搜索所有子目录， glob() 只搜索当前目录
• (3) 将 yield file.name 改为 yield file 或 yield str(file)

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
def memoize(func):
    cache = {}
    def wrapper(n):
        if n not in cache:
            cache[n] = func(n)
        return cache[n]
    return wrapper
@memoize
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n-1)
print(factorial(5))
```

问题：

- (1) 两次调用 factorial(5) 的输出是什么？
- (2) 这个装饰器实现了什么功能？
- (3) 第二次调用 factorial(5) 为什么更快？

答案 • (1) 两次都输出：120
• (2) 实现了记忆化（缓存），避免重复计算
• (3) 第二次直接从缓存中获取结果，不需要重新计算

解析

五、程序设计题

56 • 填空 / 简答

设计一个 Book 类和 Library 类：

Book类要求：

类要求：

- 属性：书名、作者、ISBN、是否借出
- 方法： __str__() 返回书籍信息

Library类要求：

类要求：

- 属性：图书列表

- 方法：
  add_book(book) ：添加图书
  borrow_book(isbn) ：借书
  return_book(isbn) ：还书
  available_books ：使用 @property 返回可借图书列表
  __len__() ：返回图书总数

创建图书馆实例，添加至少3本书，演示借书和还书功能。

答案

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.is_borrowed = False
    def __str__(self):
        status = "已借出" if self.is_borrowed else "可借"
        return f"《{self.title}》- {self.author} (ISBN: {self.isbn}) [{status}]"
class Library:
    def __init__(self):
        self.books = []
    def add_book(self, book):
        self.books.append(book)
        print(f"已添加图书：{book.title}")
    def borrow_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if not book.is_borrowed:
                book.is_borrowed = True
                print(f"成功借出：{book.title}")
                return True
                else:
                print(f"《{book.title}》已被借出")
                return False
        print("未找到该图书")
        return False
    def return_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if book.is_borrowed:
                book.is_borrowed = False
                print(f"成功归还：{book.title}")
                return True
                else:
                print(f"《{book.title}》未被借出")
                return False
        print("未找到该图书")
        return False
    @property
```

```
    def available_books(self):
        return [book for book in self.books if not book.is_borrowed]
    def __len__(self):
        return len(self.books)
# 使用示例
library = Library()
library.add_book(Book("Python编程", "张三", "001"))
library.add_book(Book("数据分析", "李四", "002"))
library.add_book(Book("机器学习", "王五", "003"))
print(f"\n图书馆共有{len(library)} 本书")
print("\n可借图书：")
for book in library.available_books:
    print(book)
library.borrow_book("001")
library.return_book("001")
```

解析

57 • 填空 / 简答

编写以下函数：

1. read_numbers(filename) ：生成器函数，逐行读取文件中的数字
2. running_average(numbers) ：生成器函数，计算移动平均值
3. 使用装饰器 @timer 测量函数执行时间

要求：

- 使用 yield 实现惰性计算
- 处理文件不存在的异常
- 使用 pathlib 处理文件路径

答案

```
from pathlib import Path
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} 执行时间: {end-start:.4f}秒")
        return result
    return wrapper
def read_numbers(filename):
    path = Path(filename)
    try:
        if not path.exists():
            raise FileNotFoundError(f"文件{filename} 不存在")
        with path.open('r') as f:
            for line in f:
                try:
                yield float(line.strip())
                except ValueError:
                continue
```

```
    except Exception as e:
        print(f"读取文件错误: {e}")
def running_average(numbers):
    total = 0
    count = 0
    for num in numbers:
        total += num
        count += 1
        yield total / count
@timer
def process_data(filename):
    numbers = read_numbers(filename)
    averages = running_average(numbers)
    return list(averages)
# 使用示例
# 创建测试文件
test_file = Path('numbers.txt')
test_file.write_text('10\n20\n30\n40\n50')
result = process_data('numbers.txt')
print(f"移动平均值: {result}")
```

解析

58 • 填空 / 简答

使用Pandas完成以下任务：

创建销售数据DataFrame（日期、产品、销量、金额）：

- 计算每个产品的总销量和总金额
- 找出销量最高的产品
- 按月份统计销售额
- 使用 apply() 计算折扣后金额（9折）
- 使用列表推导式筛选销量大于100的记录
- 将结果保存为Excel文件

答案

```
import pandas as pd
from datetime import datetime, timedelta
# 创建销售数据
dates = [datetime(2024, 1, 1) + timedelta(days=i) for i in range(10)]
data = {
    '日期': dates,
    '产品': ['A', 'B', 'A', 'C', 'B', 'A', 'C', 'B', 'A', 'C'],
    '销量': [120, 80, 150, 90, 110, 130, 95, 100, 140, 85],
    '金额': [1200, 1600, 1500, 1800, 2200, 1300, 1900, 2000, 1400, 1700]
}
df = pd.DataFrame(data)
# 计算每个产品的总销量和总金额
product_stats = df.groupby('产品').agg({
    '销量': 'sum',
    '金额': 'sum'
})
```

```
print("产品统计：")
print(product_stats)
# 找出销量最高的产品
top_product = df.groupby('产品')['销量'].sum().idxmax()
print(f"\n销量最高的产品: {top_product}")
# 按月份统计销售额
df['月份'] = df['日期'].dt.to_period('M')
monthly_sales = df.groupby('月份')['金额'].sum()
print(f"\n月度销售额:\n{monthly_sales}")
# 计算折扣后金额
df['折扣后金额'] = df['金额'].apply(lambda x: x * 0.9)
# 筛选销量大于100的记录
high_sales = [row for _, row in df.iterrows() if row['销量'] > 100]
print(f"\n销量大于100的记录数: {len(high_sales)}")
# 保存为Excel
df.to_excel('sales_analysis.xlsx', index=False)
print("\n数据已保存到 sales_analysis.xlsx")
```

解析

59 • 填空 / 简答

使用 pathlib 实现以下功能：

1. create_project_structure(base_path, structure) ：
- 根据字典结构创建项目目录
- 例如： {'src': ['main.py', 'utils.py'], 'tests': []}
2. find_large_files(directory, size_mb) ：
- 使用生成器返回大于指定大小的文件
- 返回文件路径和大小
3. backup_files(source, destination, pattern) ：
- 复制匹配模式的文件到目标目录
- 使用 Path 对象操作

答案

```
from pathlib import Path
import shutil
def create_project_structure(base_path, structure):
    base = Path(base_path)
    base.mkdir(parents=True, exist_ok=True)
    for folder, files in structure.items():
        folder_path = base / folder
        folder_path.mkdir(exist_ok=True)
        for file in files:
            file_path = folder_path / file
            file_path.touch()
    print(f"项目结构已创建在: {base}")
def find_large_files(directory, size_mb):
    path = Path(directory)
    size_bytes = size_mb * 1024 * 1024
    for file in path.rglob('*'):
        if file.is_file():
```

```
            file_size = file.stat().st_size
            if file_size > size_bytes:
                yield (file, file_size / 1024 / 1024)
def backup_files(source, destination, pattern):
    source_path = Path(source)
    dest_path = Path(destination)
    dest_path.mkdir(parents=True, exist_ok=True)
    count = 0
    for file in source_path.glob(pattern):
        if file.is_file():
            shutil.copy2(file, dest_path / file.name)
            count += 1
    print(f"已备份{count} 个文件到{destination}")
# 使用示例
structure = {
'src': ['main.py', 'utils.py'],
'tests': ['test_main.py'],
'docs': []
}
create_project_structure('my_project', structure)
# 查找大文件
# for file, size in find_large_files('.', 1):
# print(f"{file}: {size:.2f} MB")
# 备份文件
# backup_files('src', 'backup', '*.py')
```

解析

60 • 填空 / 简答

实现以下高级函数功能：

1. compose(*functions) ：函数组合装饰器
- 将多个函数组合成一个
- 从右到左执行

2. retry(times, exceptions) ：重试装饰器
- 函数失败时自动重试
- 指定重试次数和捕获的异常

3. curry(func) ：柯里化装饰器
- 将多参数函数转换为单参数函数链

演示这些装饰器的使用。

答案

```
from functools import wraps
import time
def compose(*functions):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = func(*args, **kwargs)
            for f in reversed(functions):
                result = f(result)
```

```
            return result
        return wrapper
    return decorator
def retry(times=3, exceptions=(Exception,)):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                return func(*args, **kwargs)
                except exceptions as e:
                if attempt == times - 1:
                raise
                print(f"尝试{attempt + 1} 失败: {e}")
                time.sleep(1)
        return wrapper
    return decorator
def curry(func):
    @wraps(func)
    def wrapper(x):
        def inner(y):
            return func(x, y)
        return inner
    return wrapper
# 使用示例
@compose(lambda x: x * 2, lambda x: x + 10)
def base_value():
    return 5
print(f"组合函数结果: {base_value()}")
# (5 + 10) * 2 = 30
@retry(times=3, exceptions=(ValueError,))
def unstable_function():
    import random
    if random.random() < 0.7:
        raise ValueError("随机错误")
    return "成功"
try:
    result = unstable_function()
    print(f"重试函数结果: {result}")
except ValueError:
    print("所有重试都失败了")
@curry
def add(x, y):
    return x + y
add_5 = add(5)
print(f"柯里化结果: {add_5(3)}")
                # 8
```

程序设计题详细参考答案

```
题目1：图书管理系统（完整实现）
"""
图书管理系统
功能：图书管理、借阅管理、统计查询
使用面向对象设计和装饰器模式
"""
from datetime import datetime, timedelta
from functools import wraps
import json
def log_operation(func):
    """日志装饰器 - 记录所有操作"""
    @wraps(func)
    def wrapper(self, *args, **kwargs):
        operation = func.__name__
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        result = func(self, *args, **kwargs)
        print(f"[{timestamp}] 操作: {operation}, 参数: {args}")
        return result
    return wrapper
class Book:
    """图书类"""
    def __init__(self, book_id, title, author, isbn, total_copies=1):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.isbn = isbn
        self.total_copies = total_copies
        self.available_copies = total_copies
        self.borrowed_by = []
    # 借阅记录
    def is_available(self):
        """检查是否有可借阅的副本"""
        return self.available_copies > 0
    def __str__(self):
        return f"《{self.title}》- {self.author} (可借: {self.available_copies}/{self.total
    def __repr__(self):
        return f"Book({self.book_id}, {self.title}, {self.author})"
class Member:
    """会员类"""
    def __init__(self, member_id, name, max_books=3):
        self.member_id = member_id
        self.name = name
        self.max_books = max_books
        self.borrowed_books = []
        # 当前借阅的图书
        self.borrow_history = []
    # 借阅历史
    def can_borrow(self):
```

```
        """检查是否可以借书"""
        return len(self.borrowed_books) < self.max_books
    def __str__(self):
        return f"会员: {self.name} (已借: {len(self.borrowed_books)}/{self.max_books})"
class BorrowRecord:
    """借阅记录类"""
    def __init__(self, book, member, borrow_date=None):
        self.book = book
        self.member = member
        self.borrow_date = borrow_date or datetime.now()
        self.due_date = self.borrow_date + timedelta(days=30)
        self.return_date = None
    def is_overdue(self):
        """检查是否逾期"""
        if self.return_date:
            return False
        return datetime.now() > self.due_date
    def days_overdue(self):
        """计算逾期天数"""
        if not self.is_overdue():
            return 0
        return (datetime.now() - self.due_date).days
    def __str__(self):
        status = "已归还" if self.return_date else ("逾期" if self.is_overdue() else "借阅中
        return f"{self.member.name} 借阅 《{self.book.title}》 - {status}"
class Library:
    """图书馆管理系统"""
    def __init__(self, name):
        self.name = name
        self.books = {}
        # book_id -> Book
        self.members = {}
        # member_id -> Member
        self.borrow_records = []
    @log_operation
    def add_book(self, book):
        """添加图书"""
        if book.book_id in self.books:
            # 如果图书已存在，增加副本数
            existing_book = self.books[book.book_id]
            existing_book.total_copies += book.total_copies
            existing_book.available_copies += book.total_copies
            print(f"增加《{book.title}》副本，现有{existing_book.total_copies} 本")
        else:
            self.books[book.book_id] = book
            print(f"成功添加图书: {book}")
    @log_operation
    def add_member(self, member):
```

```
        """添加会员"""
        if member.member_id in self.members:
            print(f"会员{member.name} 已存在")
            return False
    self.members[member.member_id] = member
    print(f"成功添加会员: {member}")
    return True
@log_operation
def borrow_book(self, member_id, book_id):
    """借书"""
    # 检查会员和图书是否存在
    if member_id not in self.members:
        print("会员不存在")
        return False
    if book_id not in self.books:
        print("图书不存在")
        return False
    member = self.members[member_id]
    book = self.books[book_id]
    # 检查借阅条件
    if not member.can_borrow():
        print(f"{member.name} 已达到借阅上限")
        return False
    if not book.is_available():
        print(f"《{book.title}》暂无可借副本")
        return False
    # 执行借阅
    record = BorrowRecord(book, member)
    self.borrow_records.append(record)
    book.available_copies -= 1
    book.borrowed_by.append(member_id)
    member.borrowed_books.append(book_id)
    member.borrow_history.append(record)
    print(f"借阅成功: {member.name} 借阅《{book.title}》")
    print(f"应还日期: {record.due_date.strftime('%Y-%m-%d')}")
    return True
@log_operation
def return_book(self, member_id, book_id):
    """还书"""
    if member_id not in self.members or book_id not in self.books:
        print("会员或图书不存在")
        return False
    member = self.members[member_id]
    book = self.books[book_id]
    if book_id not in member.borrowed_books:
        print(f"{member.name} 未借阅《{book.title}》")
        return False
    # 查找借阅记录
```

```
    record = None
    for r in reversed(self.borrow_records):
        if r.member.member_id == member_id and r.book.book_id == book_id and not r.ret
            record = r
            break
    if not record:
        print("未找到借阅记录")
        return False
    # 执行归还
    record.return_date = datetime.now()
    book.available_copies += 1
    book.borrowed_by.remove(member_id)
    member.borrowed_books.remove(book_id)
    # 检查是否逾期
    if record.is_overdue():
        days = record.days_overdue()
        fine = days * 0.5
        # 每天罚款0.5元
        print(f"归还成功（逾期{days} 天，罚款{fine:.2f} 元）")
    else:
        print(f"归还成功: {member.name} 归还《{book.title}》")
    return True
def search_books(self, keyword):
    """搜索图书"""
    results = []
    keyword_lower = keyword.lower()
    for book in self.books.values():
        if (keyword_lower in book.title.lower() or
            keyword_lower in book.author.lower() or
            keyword_lower in book.isbn):
            results.append(book)
    return results
def get_overdue_books(self):
    """获取逾期图书列表"""
    overdue = []
    for record in self.borrow_records:
        if record.is_overdue():
            overdue.append(record)
    return overdue
def generate_report(self):
    """生成统计报告"""
    print("\n" + "="*60)
    print(f"{self.name} - 统计报告".center(60))
    print("="*60)
    print(f"\n图书总数: {len(self.books)} 种")
    total_copies = sum(book.total_copies for book in self.books.values())
    available_copies = sum(book.available_copies for book in self.books.values())
    print(f"藏书总量: {total_copies} 本 (可借: {available_copies} 本)")
```

```
    print(f"\n会员总数: {len(self.members)} 人")
    active_members = sum(1 for m in self.members.values() if m.borrowed_books)
    print(f"活跃会员: {active_members} 人")
    print(f"\n借阅记录: {len(self.borrow_records)} 条")
    active_borrows = sum(1 for r in self.borrow_records if not r.return_date)
    print(f"当前借阅: {active_borrows} 本")
        overdue = self.get_overdue_books()
        if overdue:
            print(f"\n逾期图书: {len(overdue)} 本")
            for record in overdue[:5]:
                # 显示前5条
                print(f" - {record}")
        print("="*60 + "\n")
# 使用示例
if __name__ == "__main__":
    # 创建图书馆
    library = Library("市图书馆")
    # 添加图书
    library.add_book(Book("B001", "Python编程", "张三", "978-1234567890", 3))
    library.add_book(Book("B002", "数据结构", "李四", "978-0987654321", 2))
    library.add_book(Book("B003", "算法导论", "王五", "978-1111111111", 1))
    # 添加会员
    library.add_member(Member("M001", "小明"))
    library.add_member(Member("M002", "小红"))
    library.add_member(Member("M003", "小刚"))
    # 借书
    print("\n" + "="*60)
    print("借阅操作")
    print("="*60)
    library.borrow_book("M001", "B001")
    library.borrow_book("M001", "B002")
    library.borrow_book("M002", "B001")
    # 搜索图书
    print("\n" + "="*60)
    print("搜索图书")
    print("="*60)
    results = library.search_books("Python")
    print(f"搜索 'Python' 的结果:")
    for book in results:
    print(f"
            {book}")
# 还书
print("\n" + "="*60)
print("归还操作")
print("="*60)
library.return_book("M001", "B001")
# 生成报告
library.generate_report()
```

设计要点：
1. 多类协作
多类协作：Book、Member、BorrowRecord、Library四个类各司其职
2. 装饰器应用
装饰器应用：使用@log_operation记录所有操作
3. 业务逻辑
业务逻辑：完整的借还书流程，包括逾期检查
4. 数据管理
数据管理：使用字典存储，便于快速查找
5. 扩展性
扩展性：易于添加新功能（如预约、续借等）
题目2：日志分析工具（完整实现）

```
"""
日志分析工具
功能：解析日志文件、统计分析、生成报告
使用生成器处理大文件，Pandas进行数据分析
"""
import re
from pathlib import Path
from datetime import datetime
from collections import Counter, defaultdict
import pandas as pd
class LogParser:
    """日志解析器"""
    # 常见日志格式的正则表达式
    PATTERNS = {
        'apache': r'(\S+) \S+ \S+ \[([\w:/]+\s[+\-]\d{4})\] "(\S+) (\S+) (\S+)" (\d{3}) (\
        'nginx': r'(\S+) - - \[(.*?)\] "(\w+) (.*?) HTTP/.*?" (\d+) (\d+)',
        'python': r'(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}),\d+ - (\w+) - (.+)',
        'custom': r'\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] (\w+): (.+)'
    }
    def __init__(self, log_format='custom'):
        self.log_format = log_format
        self.pattern = re.compile(self.PATTERNS.get(log_format, self.PATTERNS['custom']))
    def parse_line(self, line):
        """解析单行日志"""
        match = self.pattern.match(line.strip())
        if not match:
            return None
        if self.log_format == 'apache':
            return {
                'ip': match.group(1),
                'timestamp': match.group(2),
                'method': match.group(3),
                'url': match.group(4),
                'protocol': match.group(5),
                'status': int(match.group(6)),
                'size': int(match.group(7))
```

```
            }
        elif self.log_format == 'python':
            return {
                'timestamp': match.group(1),
                'level': match.group(2),
                'message': match.group(3)
            }
        elif self.log_format == 'custom':
            return {
                'timestamp': match.group(1),
                'level': match.group(2),
                'message': match.group(3)
            }
        return None
    def parse_file(self, filepath):
        """
        生成器：逐行解析日志文件
        :param filepath: 日志文件路径
        :yield: 解析后的日志字典
        """
        try:
            with open(filepath, 'r', encoding='utf-8') as f:
                for line_num, line in enumerate(f, 1):
                parsed = self.parse_line(line)
                if parsed:
                parsed['line_num'] = line_num
                yield parsed
        except Exception as e:
            print(f"解析文件出错: {e}")
class LogAnalyzer:
    """日志分析器"""
def __init__(self, parser):
    self.parser = parser
    self.logs = []
def load_logs(self, filepath):
    """加载日志"""
    print(f"正在加载日志: {filepath}")
    self.logs = list(self.parser.parse_file(filepath))
    print(f"成功加载{len(self.logs)} 条日志")
    return self.logs
def filter_by_level(self, level):
    """按日志级别过滤"""
    return [log for log in self.logs if log.get('level') == level]
def filter_by_time_range(self, start_time, end_time):
    """按时间范围过滤"""
    filtered = []
    for log in self.logs:
        try:
```

```
            log_time = datetime.strptime(log['timestamp'], '%Y-%m-%d %H:%M:%S')
            if start_time <= log_time <= end_time:
                filtered.append(log)
        except:
            continue
    return filtered
def count_by_level(self):
    """统计各级别日志数量"""
    levels = [log.get('level', 'UNKNOWN') for log in self.logs]
    return Counter(levels)
def count_by_hour(self):
    """按小时统计日志数量"""
    hours = defaultdict(int)
    for log in self.logs:
        try:
            timestamp = log['timestamp']
            hour = timestamp.split()[1].split(':')[0]
            hours[hour] += 1
        except:
            continue
    return dict(sorted(hours.items()))
def find_errors(self):
    """查找错误日志"""
    errors = []
    for log in self.logs:
        level = log.get('level', '').upper()
        if level in ['ERROR', 'CRITICAL', 'FATAL']:
            errors.append(log)
    return errors
def search_keyword(self, keyword):
    """搜索包含关键词的日志"""
    results = []
    keyword_lower = keyword.lower()
    for log in self.logs:
        message = log.get('message', '').lower()
        if keyword_lower in message:
            results.append(log)
    return results
def to_dataframe(self):
    """转换为Pandas DataFrame"""
    if not self.logs:
        return pd.DataFrame()
    return pd.DataFrame(self.logs)
def generate_report(self, output_file='log_report.txt'):
    """生成分析报告"""
    report = []
    report.append("="*70)
    report.append("日志分析报告".center(70))
```

```
    report.append("="*70)
    report.append(f"\n总日志条数: {len(self.logs)}")
    # 按级别统计
    level_counts = self.count_by_level()
    report.append("\n按级别统计:")
        for level, count in sorted(level_counts.items()):
            percentage = (count / len(self.logs)) * 100
            report.append(f"
                {level}: {count} ({percentage:.2f}%)")
        # 按小时统计
        hour_counts = self.count_by_hour()
        report.append("\n按小时统计:")
        for hour, count in hour_counts.items():
            report.append(f"
                {hour}:00 - {count} 条")
        # 错误日志
        errors = self.find_errors()
        report.append(f"\n错误日志: {len(errors)} 条")
        if errors:
            report.append("最近5条错误:")
            for error in errors[:5]:
                report.append(f" [{error['timestamp']}] {error.get('message', '')[:60]}")
        report.append("="*70)
        # 输出到文件
        report_text = '\n'.join(report)
        Path(output_file).write_text(report_text, encoding='utf-8')
        print(f"\n报告已保存到: {output_file}")
        print(report_text)
        return report_text
# 使用示例
if __name__ == "__main__":
    # 创建测试日志文件
    test_log = Path("test_app.log")
    log_content = """[2024-01-01 10:15:23] INFO: 应用启动成功
[2024-01-01 10:15:24] DEBUG: 加载配置文件
[2024-01-01 10:15:25] INFO: 数据库连接成功
[2024-01-01 10:20:30] WARNING: 内存使用率达到80%
[2024-01-01 10:25:45] ERROR: 数据库查询超时
[2024-01-01 10:30:12] INFO: 处理请求 /api/users
[2024-01-01 10:35:20] ERROR: 文件未找到: config.json
[2024-01-01 10:40:15] INFO: 用户登录: user123
[2024-01-01 11:10:30] WARNING: API调用频率过高
[2024-01-01 11:15:45] INFO: 定时任务执行完成
[2024-01-01 11:20:50] CRITICAL: 系统内存不足
[2024-01-01 11:25:10] INFO: 应用正常运行
"""
test_log.write_text(log_content, encoding='utf-8')
# 创建解析器和分析器
```

```
parser = LogParser(log_format='custom')
analyzer = LogAnalyzer(parser)
# 加载日志
analyzer.load_logs(test_log)
# 统计分析
print("\n" + "="*70)
print("日志级别统计")
print("="*70)
level_counts = analyzer.count_by_level()
for level, count in sorted(level_counts.items()):
    print(f"{level}: {count}")
# 查找错误
print("\n" + "="*70)
print("错误日志")
print("="*70)
errors = analyzer.find_errors()
for error in errors:
    print(f"[{error['timestamp']}] {error['level']}: {error['message']}")
# 搜索关键词
print("\n" + "="*70)
print("搜索关键词: '数据库'")
print("="*70)
results = analyzer.search_keyword('数据库')
for log in results:
    print(f"[{log['timestamp']}] {log['message']}")
# 生成报告
analyzer.generate_report()
# 转换为DataFrame进行高级分析
df = analyzer.to_dataframe()
print("\n" + "="*70)
print("DataFrame分析")
print("="*70)
print(df[['timestamp', 'level', 'message']].head())
# 清理测试文件
test_log.unlink()
```

设计要点：

1. 正则表达式

正则表达式：支持多种日志格式的解析

2. 生成器模式

生成器模式：逐行读取大文件，节省内存

3. 统计分析

统计分析：使用Counter和defaultdict进行高效统计

4. Pandas集成

集成：转换为DataFrame便于高级分析

5. 报告生成

报告生成：自动生成详细的分析报告

由于篇幅限制，其余题目的详细答案请参考第1套试卷的答案模式。核心要点：

- 题目

题目3：使用Pandas进行销售数据分析，包括数据清洗、分组聚合、时间序列分析
- 题目

题目4：实现配置管理系统，使用装饰器和上下文管理器
- 题目

题目5：高级装饰器实现（compose、retry、curry等函数式编程技巧）

评分说明：
- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析